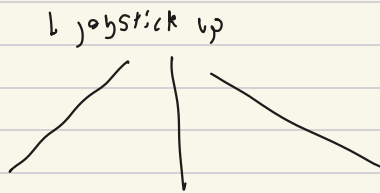
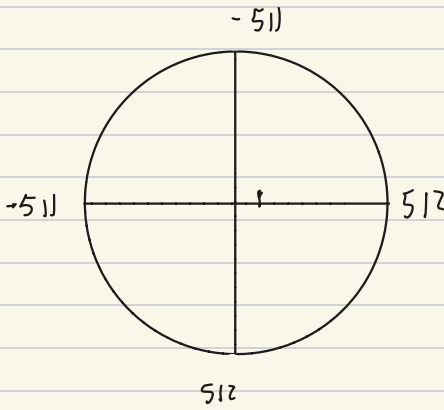
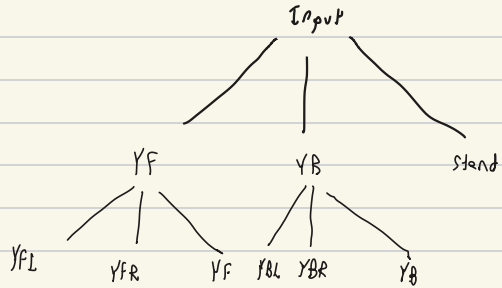
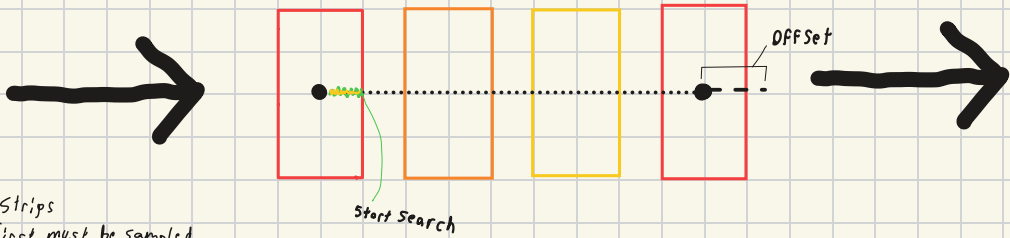


Joystick



Joystick down

Color Automation



4 Strips

- First must be sampled
- The next 3 will be random. You have to stop at sampled color.

Steps

- 1) Place robot on sample color
- 2) switch to color automation
- 3) store color currently being read with input or automatically
- 4) Move to start search when sampled color no longer detected
- 5) Start search and keep going until sampled color is found
- 6) Stop at offset time so robot is directly under the color
 - a) Figure offset out by debugging

Notes:

Use while loops to isolate all computations
 while = can't switch modes unless implemented inside
 No while = harder to work with

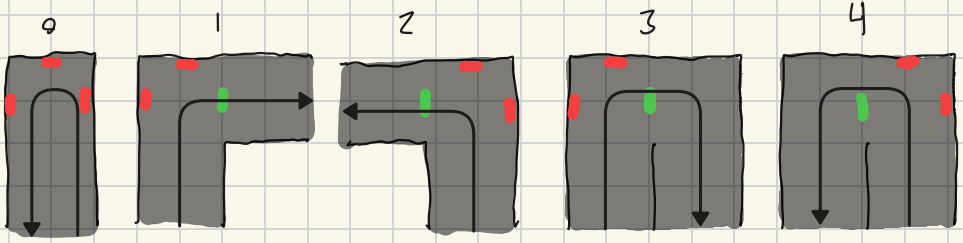
Code:

```
colorAutomation() {
    // store color being sampled
    bool checkInitial = false;
    bool colorFound = false;
    while (! (checkInitial == true || colorFound == true)) {
        colorFound = false;
        if (currentColor != sampledColor) {
            checkInitial = true;
        }
        if (currentColor == sampledColor) {
            colorFound = true;
        }
    }
    moveMotors Helper (X, 0, X, 0);
    // aft set
    delay(XXX);
    moveMotors Helper (0, 0, 0, 0);
}
```



Maze Automaten

Cases:



0: All sensors returning valve close to wall
→ 180°

1: Left & front close, right far
→ 90° right

→ keep moving until front close

2: Right & front close, left far
→ 90° left

→ keep moving forward until front close

3:
4: $\rightarrow$ Same logic

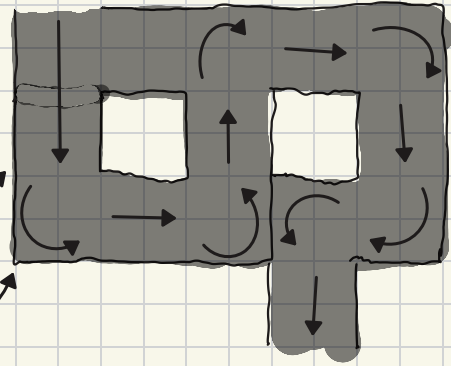
Notes:

cases 3 & 4 follow the same logic

Importance should be:

check front, if for, continue straight
will only turn when front close

Move forward until wall,
turn left if left open, or turn
right if left & right open



Doesn't have to be cases. You can
dynamically adjust based on
difference

Logic

- Green: $\text{sensor} \geq \text{turnThreshold}$
- Red: $\text{sensor} < \text{turnThreshold}$

```
Logic: sensor < turnThreshold
```

L < R? Slightly turn right

L > R? slightly turn left

$I \approx \mathbb{R}^p$ move straight

front sensor →  → Move forward

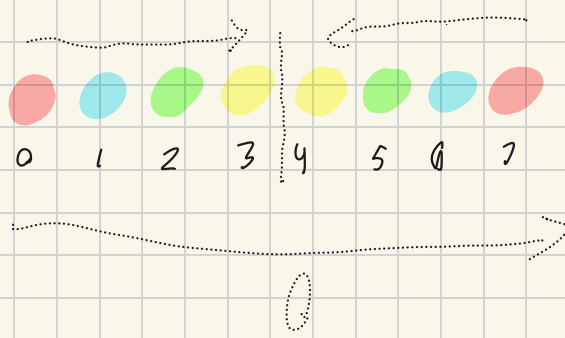
$L \text{ and } R \text{ ? } \rightarrow 180^\circ$

$L \rightarrow R \rightarrow P \rightarrow 90^\circ$ clock wise

L  B R  P $\rightarrow$ 90° clock wise

L to R $\rightarrow 90^\circ$ counter-clockwise

hug right wall



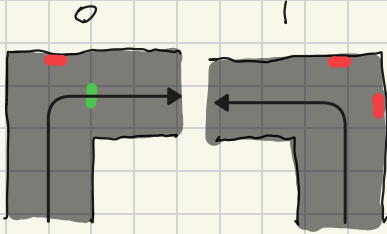
[1 1 1 1 1 1 1 1]

Maze Automation

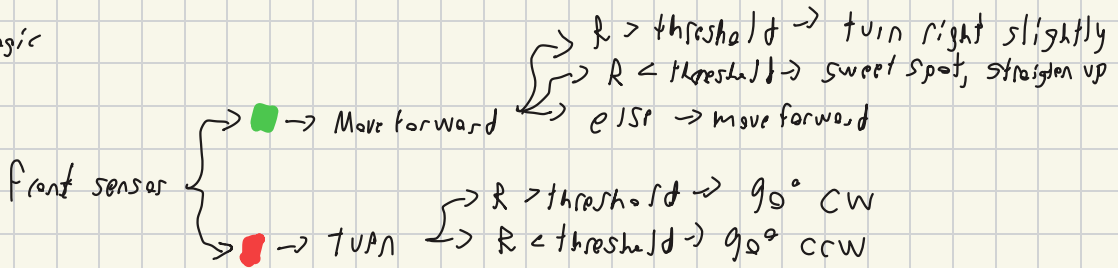
Case 0: F on wall, R not $\rightarrow 90^\circ$ right

Case 1: F on wall, R on wall $\rightarrow 90^\circ$ left

Cases:



Logic



Notes:

When is R considered close to the wall?

We need to ensure it never crosses the threshold while going straight

If right threshold set at 20cm, it should never become more than 20cm

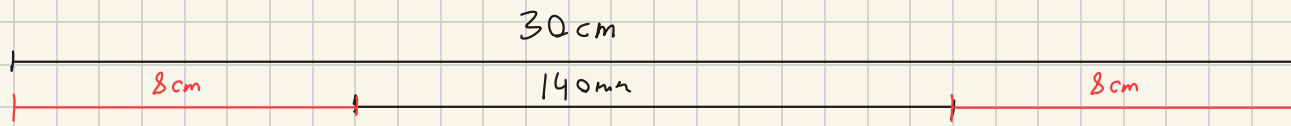
Compare C & P

If $C - P > 0$: Moving away from wall, turn right

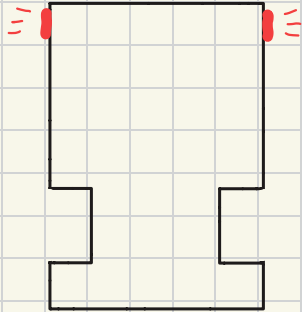
$C - P < 0$: Moving close to wall, turn left

$C - P \approx 0$: Same position, go forward

Can make everything much simpler by creating separate wall threshold for right side



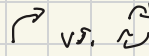
put sensor in front so it senses corner faster



L & R

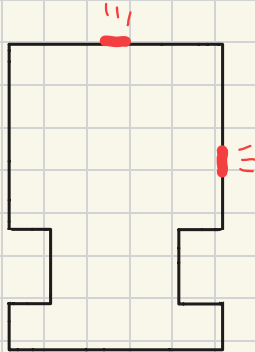
Pro: • Easier centering

• faster turning



Con: • will have to implement delay so it doesn't turn right at the end of the maze after turn, move forward until sensors are blocked again

Instead of hard coding 90°, make it rotate until sensor is blocked again



F & R

Pro: Easier logic
• no delay



Con: • More logic to center
• has to do ~ so slower turning

Note: If sensors aren't accurate at 9cm, would be better to make it skewed on one wall while following

